

MiniMCTAD: Minimalist Monte Carlo Transport Architecture Design

1st Siqing Fu

National University of Defense Technology
Changsha, China
fusiqingnudt@nudt.edu.cn

2nd Tiejun Li✉

National University of Defense Technology
Changsha, China
tjli@nudt.edu.cn

3rd Jianmin Zhang

National University of Defense Technology
Changsha, China
jmzhang@nudt.edu.cn

4th Sheng Ma

National University of Defense Technology
Changsha, China
masheng@nudt.edu.cn

5th Sheng Liu

Beihang University
Beijing, China
liu_sheng@buaa.edu.cn

Abstract—Monte Carlo (MC) method is a kind of statistical experiment method, which has been widely used in scientific research, especially in solving particle transport problems. The increase in application scale requires faster machine speeds, so we propose a novel and simple architecture, MiniMCTAD, to speed up MC Transport applications. We improve parallelism by interating more cores on a single chip, rather than improving single-threaded performance through complex on-chip designs. To achieve high-performance and low-power architecture, we design the CPU pipeline structure, execution unit, memory hierarchy and explore the design space. The most commonly used architecture simulators, gem5 and McPAT, are employed to evaluate the appropriate architecture model. The experimental results show that the performance of MiniMCTAD architecture is 1.416x that of out-of-order core architecture, and it also has less area and lower power consumption. In that same number of cores, the MiniMCTAD architecture achieves a 4.45x advantage on performance per watt and 2.78x advantage on performance per area.

Index Terms—Monte Carlo method, Transport code, Minimalism, In-order cores

I. INTRODUCTION

Since the birth of computers, their performance has continued to benefit from the miniaturization of semiconductor devices and follow Moore's Law [1]. The reduction of transistor size presents opportunities for processor performance improvement, but with it comes the complexity of the on-chip structure. Taking Intel processors as an example, p6 processors [2] in 1995 used superscalar, out-of-order execution and large memory technologies; NetBurst microarchitecture [3] in 2000 introduced the Hyper-Threading technology; Core [4] began to introduce multi-core and hardware virtualization in 2006; Sandy Bridge introduced the 256-bit Advanced Vector Extension (AVX) [5] in 2011, which was later expanded to AVX-512 [6].

However, as Intel begins to use the 10nm technology [7], the trend of Moore's Law has slowed [8]. Under these

conditions, people will no longer reap the endless benefits of complex architectural design. In [9], Leiserson discusses the sources of computer performance in the post-Moore era. He states that if we can no longer benefit from transistor size, architecture development will once again move towards domain specialization, as described by Makimoto's Wave [10]. An important aspect of domain specialization is hardware simplification. In the past, complex architectural designs have led to single-thread performance improvements, while post-Moore architects will work in the opposite direction. The result of the simplified hardware is to provide more on-chip area to connect more circuits in parallel. Fortunately, a large portion of the emerging workload includes sufficient parallelism [11], including machine learning, image processing, and MC Transport applications used in this article.

MC method, also known as a statistical test method, can be said to be a calculation method using the computer to simulate the behaviour of particles randomly. The MC method is widely used in financial engineering, Macroeconomics, Computational Physics (such as particle transport calculation, quantum thermodynamics calculation, aerodynamic calculation) and other fields. Many calculation programs use the MC method, such as MCNP, TIGER, ETRAN, GRANT. We use Mercury [12]'s agent, Quicksilver [13], as the workload. The main challenges of MC numerical simulation are colossal memory consumption, long calculation time and uncertain convergence process. However, the natural parallelism of the MC method brings opportunities to repair these problems.

There are numerous redundant and complex mechanisms in modern processors to make serial instruction streams run faster, but specific programs may have different requirements. Based on this idea, we replace the complex out-of-order processor with a more simplified core, use the design space exploration method to clarify the design parameters and architecture of the simple core, and then propose a parallel architecture, and use Quicksilver as the workload to weigh the pros and cons. The main contributions of this paper are:

- We analyze the program characteristics and control paths

Supported by the National Key Research and Development Program of China under grant No. 2018YFB0204301 and the Science and Technology Innovation project of Hunan Province (2018RS3083).

of a typical MC Transport agent program Quicksilver and describe the performance ratio of specific program segments under different branches of the program.

- We use the design space exploration method to design the pipeline structure and the memory structure, and we show the advantages over the out-of-order core. Compared with the Cortex-A15 out-of-order architecture, the performance of the Minimalism decreases by 41% under the same number of cores, but the area advantage is 6.26x, and the power advantage is 3.92x.
- We demonstrate the performance advantage by increasing parallelism. By increasing the number of cores in the MiniMCTAD architecture to eight, our architecture provides 1.41x the performance of the out-of-order architecture, while consuming only one-third of the power and one-half the area.

The rest of this paper is organized as follows: Section II introduces the basic principle of the history-based particle transport code; Section III mainly introduces the in-order core and the architecture used in this paper; Section IV introduces the experimental setup; Section V is the experimental part, which makes clear the core configuration through the design space exploration, and compares the effect of the simple core and the out-of-order core when running workloads; Section VI introduces the relevant work and finally summarizes the conclusions in Section VII.

II. PARTICLE TRANSPORT APPLICATIONS

The theory of particle transport comes from the theory of molecular motion. Boltzmann derived Boltzmann's equation describing the change of macroscopic quantities (such as energy, charge or particle number) in thermodynamic systems in the 19th century. On this basis, after discovering neutrons and the advent of computers, the stochastic simulation method, that is, the MC method, has become the primary means to solve the transport equations in microscopic particle systems. This chapter introduces the program compositions and controls flow characteristics with Quicksilver's workload.

A. History-based MC Transport Program

The MC Transport program includes an event-based method and a history-based method, and Quicksilver is a history-based program. In the process of solving the transport problem, the actual motion history of each particle in a given geometric structure is established, and then the motion state of each particle is tracked, and the sampling value is obtained by tracking. Then the estimation of a numerical characteristic is obtained by using statistical methods.

The history of a particle's motion refers to how a particle emits from a source, undergoes various reactions in a medium, and ends up tracking or dying. As an agent, Quicksilver simplifies physical processes and reactions as much as possible while maintaining consistent control flow with the parent program. In tracking, particles may cross facets, scatter, fission. In addition, when the particle escape, absorption, or simulation time reaches an endpoint, the tracking of the particle is ended.

The characteristic values to be obtained may be flux density, dose rate, precipitation energy.

When the geometry, material composition and initial source parameters of the problem are determined, the solution of the transport equation is uniquely determined. Figure 1 illustrates the five steps of the particle motion history, and they keep looping until certain preset conditions are met. In Quicksilver, the time step for the tracking simulation can be set, and the simulation will end after several time steps. Among all the particles, neutrons have the highest research maturity, and the reaction cross-section of each particle is different, but the solution process is the same. We will describe the cross-section settings for Quicksilver below.

B. Quicksilver

The MC Transport code Quicksilver is an agent for Mercury developed as a prototype to test and explore potential programming models and design options for Mercury. As an agent, Quicksilver accurately captures the control flow of the parent program Mercury, including the branches resulting from random sampling and the memory access patterns associated with reading the cross-section table.

Quicksilver executes the three functions `cycleInit()`, `cycleTracking()`, and `cycleFinalize()` in turn within each time step, and the run time of the three functions under different scale inputs is shown in the Figure 2. The processor is the AMD Ryzen5 PRO 2600 Six-Core Processor. The operating system is Ubuntu 20.04 LTS. The program includes three stages: initializing particle information, tracking particle transport, updating data and releasing cache. It can be seen that with the increase of particle number, the program time occupied by tracking gradually decreases, but it is still more than 95% in the experimental range. In addition, Quicksilver only parallelizes the tracking function, so this paper focuses on the performance of the `cycleTracking()`.

Algorithm 1 from Quicksilver [13] shows the structure of the `cycleTracking()` function. This function tracks each particle in parallel, performing different events on them. In the process of tracking a particle, the `Segment_Outcome()` function is first called to decide which branch to enter. The crossing, collision, or direct entry into census are performed respectively, and finally, the loop processing is determined whether to continue

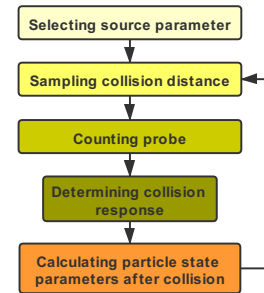


Fig. 1. The basic flow of MC Transport applications in history-based method.

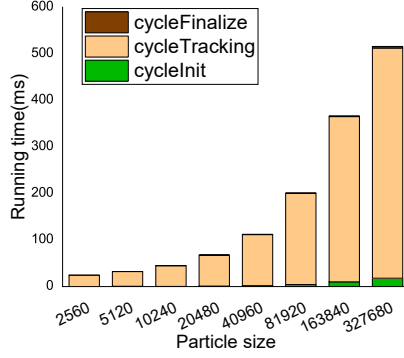


Fig. 2. The run time of the three primary functions of the Quicksilver program at different input size.

according to whether the particle escapes or is absorbed. The running time of different branches of the program during execution is shown in Figure 3. Segment_Outcome() takes up much runtime for each loop. In crossing and census events, event execution takes little time, with Segment_Outcome() accounting for more than 92%. The collision event itself also has several branches in collision events, so its running time is uncertain. Segment Outcome() accounts for 48% of the time while the branch enters a collision, and the collision event itself accounts for 24%. After calling the collision event, the control flow randomly enters the absorption, scattering and fission events, and some functions still account for the majority of the program running time.

C. Input Scale

The Quicksilver code supports accepting input parameters from the command line or an input file. Due to the diversity of MC Transport problems, the quicksilver code also supports as many input options as possible with flexibility. Table I lists some of the vital input options and the input scale values we used.

Algorithm 1 Basic structure of cycleTracking() function

Input: *Particlebank*

```

for all particles do
  while !absorbed or !incensus or !escaped do
    segmentoutcome = Segment_Outcome()
    if segmentoutcome == facet then
      facet_event()
    end if
    if segmentoutcome == collision then
      collision_event()
    end if
    if segmentoutcome == census then
      census_event()
    end if
  end while
end for

```

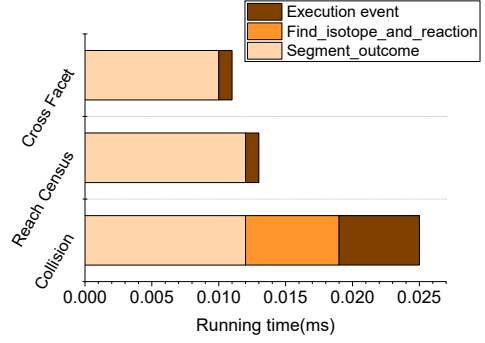


Fig. 3. Run time for multiple branches during particle tracking.

The parameter dt represents the time step, that is, the time scale of one simulation to the end. If dt is too small, it will cause many particles to enter the census directly during tracking. The parameters x , y and z represent the size of the simulation. Quicksilver simplifies the geometric range of particle transport to a cube, and proper planning of the geometric scale and particle number will allow representative results to be obtained while running the simulator as quickly as possible. Cross-section data for the material are provided by the proportions of absorption, fission, scattering, and cross-section data. The cross-section ratio reflects the probability of several collision events, while $nuBar$ is the number of prompt fission neutrons, indicating the particle generation of fission events. The cross-section data under actual conditions vary with energy, which is reduced to a 4th-degree polynomial controlled by five variables in Quicksilver. Quicksilver was provided as part of the benchmark program Coral2, and since the input parameter settings should conform to the physical laws, the inputs in Table I were taken based on the benchmark program input and in combination with the benchmark program's recommendations on the input scale.

III. CORE ARCHITECTURE

We model the MiniMCTAD architecture in the gem5 simulator [14], which is a MC Transport acceleration architecture based on the idea of simplification. Three in-order CPU implementations are provided in gem5: AtomicSimpleCPU, TimingSimpleCPU, and MinorCPU. MinorCPU is a comprehensive and detailed in-order CPU model, and we refer to it

TABLE I
SOME IMPORTANT INPUT PARAMETERS FOR QUICKSILVER

Parameter		Values
Size of simulation(cm)	x	16
	y	16
	z	16
Number of particles		163840
CrossSection ratio	Absorption	0.04
	Fission	0.05
	Scattering	1
CrossSection parametr	nuBar	1.6
Simulation parameter	dt(s)	2e-9

to design a simple in-order core compatible with the ARM instruction set.

A. Pipeline

The in-order pipeline provides four instruction stages: Fetch1, Fetch2, Decode, and Execute. Figure 4 shows the pipeline stages. Table II lists some critical parameters for each stage of the pipeline. We describe these instruction phases below.

a) *Fetch1*: The fetch stage is broken down into two parts, with fetch1 fetching the cache line from the icache and passing it to fetch2, which breaks it down into instructions. When the cache line size is determined, we can determine the number of fetches at a time, and we also focus on the latency and reverse latency between the two fetch stages.

b) *Fetch2*: Fetch2 receives the incoming cache line from the previous stage, stores it in a buffer, decomposes the cache line into individual instructions, and passes it to the decode stage. Branch prediction is also implemented in fetch2, and the predicted branch instruction is output in reverse to fetch1. Since MC Transport program involves many random branching processes, we need to verify the effect of the branch predictor, so we also configure a pipeline structure without branch prediction in modelling. The trade-off between the two will be shown in the experimental section.

c) *Decode*: The decode stage decomposes the instruction into micro-operations. We focus on input buffer size, decode-to-execute delay, and decode width.

d) *Execute*: The execution phase involves the execution of decoded instructions and memory access. The functional units involved in instruction execution are described in the next section. The width of the input execution phase, the size of the input buffer, and the number of memory accesses issued affect the performance of this phase.

The pipeline structure of MinorCPU is divided into four instruction stages, and the pipeline depth is realized by proportional adjustment simulation of instruction delay. Modern processors tend to have deep pipeline stages, which have the advantage of pursuing higher frequencies for high performance but have the negative effect of excessive overhead in the area and power consumption. The MiniMCTAD architecture aims at the small area and low power consumption of a single core, so the shallowest number of pipeline stages is adopted, and the four-level pipeline execution instruction stage is maintained.

According to our assumption, we want the single-threaded structure to be the most simplified, so each instruction stage's input width and buffer size are set as small as possible. The

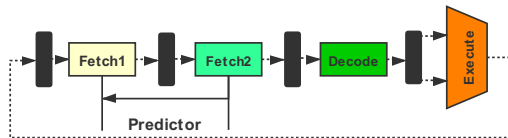


Fig. 4. The four-stage instruction pipeline of the MinorCPU.

corresponding values are listed in Table II. The exploration of the design space of the corresponding pipeline structure will be fully explained in the experimental part.

B. Branch Prediction

The branch prediction process is completed in the fetch1 and fetch2 instruction stages of the in-order pipeline. We consider whether to delete the branch predictor and the choice of different branch predictors in the design.

Considering the area, we use McPAT to calculate the area results for the Cortex-A7 in-order core modelled in gem5. The area of the whole processor is $3.32mm^2$, the area of the single core is $0.45mm^2$, and the area of the tournament branch predictor is $0.0016mm^2$, accounting for only 0.35% of the area of the single core. Even if there is no significant performance difference with or without a branch predictor, we are still willing to keep it in design with a small area cost. In selecting predictor, we also hypothesize that the most straightforward Bimode Predictor can bring good results based on the consideration of simplification. Also, in the experimental section, we will compare the performance of the architecture before and after deleting the branch predictor and compare the performance differences of different kinds of predictors to prove our design.

C. Functional Unit

The gem5 simulator supports the configuration of various functional units, which support their operation classes to execute corresponding instructions. At the same time, the operation delay of various operations can also be adjusted accordingly. Figure 5 illustrates the use of functional units for several core models. The Cortex-A15 is configured with more functional units to cope with pipeline congestion caused by occupied components. The MiniMCTAD core will exchange less functional units for area advantage, which provides an opportunity to improve the parallelism.

In addition, for the ARMv8 instruction set, pretty many instructions are not required in a domain-specific computer.

TABLE II
SOME PIPELINE PARAMETER AND DEFAULT VALUES

Instruction Stage	Parameter	Default Value
Fetch1	fetch1FetchLimit	1
	fetch1LineSnapWidth	0
	fetch1LineWidth	0
	fetch1ToFetch2ForwardDelay	1
	fetch1ToFetch2BackwardDelay	0
Fetch2	fetch2InputBufferSize	2
	fetch2ToDecodeForwardDelay	1
Decode	decodeInputBufferSize	3
	decodeToExecuteForwardDelay	1
	decodeInputWidth	1
Execute	executeInputWidth	1
	executeInputBufferSize	7
	executeIssueLimit	1
	executeMemoryIssueLimit	1
	executeCommitLimit	1
	executeMemoryCommitLimit	1

Trimming some redundant instructions will also yield significant area gains. However, in the process of performance simulation, gem5 only simulates the execution of various instructions in function and does not involve area modelling. McPAT [15] also has no corresponding mechanism to support it. Therefore, the results of this paper will only be better after combining instruction set simplification.

D. Cache Hierarchy

Cache occupies a significant portion of the on-chip area and also significantly affects performance. In this paper [16], Qureshi removes L2 cache and switches DDR4 to HBM2 when the L2 cache miss rate is very high, and the performance of next generation sequencing applications is significantly improved. However, the cache is usually the key point to improve performance. Since the trace phase does not involve a large amount of data reading and writing, we think that the two-level cache can support data reading and write effectively without wasting on-chip space, so we still choose the two-level cache structure.

In general, the cache can configure prefetcher to increase speed. In the tracking phase, the existence of random branches will inevitably affect the data locality. However, branches should be coarse-grained to affect the prefetch effect of the L2 cache, but the L1 cache should be able to function. So our L1 cache uses direct prefetch, and the L2 cache does not prefetch. Since the L1 cache is already burdensome to run the program, it retains a size of 32 kB. The L2 cache is more prominent and takes up more area, and we will define the size of the L2 cache in the lab. Extending this, we will add more new tweaks to the storage system in the future, such as the appropriate application of scratch pad memories (SPM) [17] to better leverage memory access features.

IV. EXPERIMENTAL SETUP

In order to evaluate the area, power and performance of our MiniMCTAD architecture when running the MC Transport code, we modelled the entire system using the gem5 simulator (version 20.01) in conjunction with the McPAT simulator (version 1.3). We generated the McPAT input file based on the gem5 statistics and configuration. The machine on which we

run the simulator uses the AMD Ryzen5 PRO 2600 Six-Core Processor, and the operating system is Ubuntu 20.04 LTS. In the experiment, the FS mode is mainly used. The version of the mirror installation operating system started by FS mode is Ubuntu 16.04 LTS. The Quicksilver is compiled by GCC (version 5.4.0) in QEMU (version 5.1.0), and the compilation option is OpenMP (version 4.5) parallel. Furthermore, the technology node is 28 nm.

Our modelling parameters are specified in Table III. In the experiment, we have configured several architectural models for comparative experimentation, which only involved the difference between the number of cores used and the size of the L2 cache.

A. Cortex-A15 Big Core Experimental Setup

The core is configured as Cortex-A15 core, and the model is inherited from O3CPU. It is a high-performance out-of-order core with a 15-level pipeline modelled in gem5, which is regarded as representing a big core in a specific period. Each tile is configured with four cores, corresponding to a shared L2 cache, and then each tile is connected to memory through the on-chip network. It should be mentioned that only one tile is configured in the experiment because of the natural parallel characteristics of MC Transport code and the complete parallel of tracking program phase.

B. 4-core MiniMCTAD Core Configuration

The core configuration is the MiniMCTAD core designed in Section.III, in which the main structural parameters and the size of L2 cache will be specified in the experiment. Other architecture configurations are the same as those of the out-of-order core architecture, and the two groups are directly compared to reflect the differences in performance, area and power consumption.

C. 8-core MiniMCTAD Core Configuration

In order to verify the scalability of on-chip multi-core integration, further experiments need to integrate more MiniMCTAD cores. In this paper, taking 8-core as an example, we comprehensively compare the complex core architecture and verify the results. At the same time, the L2 cache shared by each tile is also adjusted to 2.

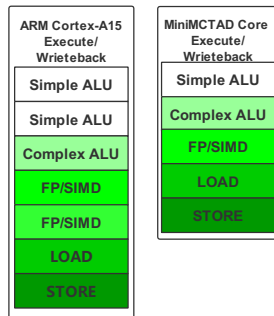


Fig. 5. Function units used by big out-of-order cores, little in-order cores, and MiniMCTAD cores.

TABLE III
EXPERIMENTAL MODELING PARAMETERS

Processing Element	2.2 GHz clock speed, 4 or 8 PEs per tile active in experiment
L1 cache	32kB, cache line size 64 icache: 2-way set-associative, privated, dcache: 4-way set-associative, privated,
L2 cache	2MB, 1MB, 512kB or 256kB each tile, 8-way set-associative, shared replacement policy is RandomRP
Main Memory	4GB, HBM 2.0 with 8 channels

V. EXPERIMENTAL RESULTS

In Section III and Section IV, we introduced the architecture of the core of minimalism and analyzed some configurations of the pipeline and storage hierarchy. In this section, We will start with some validation tests of the design details in the architecture, and then we tested the performance, power, and area differences between the MiniMCTAD architecture and the big out-of-order core on the architecture and lab setup we have already described.

A. Design Space Exploration

a) *Pipeline Structure*: We adjust the important parameters of the pipeline architecture to observe its impact on performance, power consumption, and area.

- **Effect of Pipeline Parameters on Performance**

This process is time-consuming because each modification to the pipeline parameters involves restarting FS mode and running Quicksilver fully. In order to explore the design space quickly, we use SE mode to run the DAXPY program as the workload, which briefly reflects the impact of various pipeline parameter modifications on pipeline performance.

Due to the excessive number of variables, we summarize some general rules from Figure 6. For example, some buffer size and queue size are limited by the input width, and it is not very meaningful to increase them after reaching a specific size. Another example is that the input width of each stage is restricted to each other, and the benefit is limited by simply adding one. Of course, we need to avoid excessive performance degradation, so the key is to see the impact of these parameter adjustments on the power and area.

- **Effect of Pipeline Parameters on Area and Power**

The power and area simulations are performed in McPAT, and the input parameters are based on the statistical output of gem5. Here we describe the area and power consumption of pipelines in several group configurations. McPAT's initial XML template is configured with a 5-stage pipeline that does not share integer and floating-point. Based on this, we make several adjustments: the pipeline uses a 4-stage depth and shares integer and floating-point units, reduces the input width of each instruction stage, increases the size of the instruction buffer, and integrates the configuration. The results are shown in Figure 7.

Pipeline simplification and input width reduction reduce power and area by more than 20% – 30% in total, while changes of buffer size are not sensitive. Combined with the influence of these parameters, it is reasonable for us to reduce the pipeline input width as much as possible and halve the buffer size.

b) *Branch Predictor*: We modify the source code to delete all the return values of the branch prediction function in the fetch2 stage so that the pipeline no longer has the branch prediction function. Figure 8 compares the performance difference between running Quicksilver when the MiniMCTAD

TABLE IV
THE IMPACT OF CACHE PREFETCHER ON PERFORMANCE

Component	Prefetcher	Non-prefetcher
dcache	1.13	1
l2cache	1	1.17

core has a branch structure and when the branch structure is removed. Branch prediction still plays a role for the program which has a lot of loop control and random branches, and removing the prediction structure will reduce performance by 32%.

Considering our previous analysis, the branch predictor does not take up much area, so it should be retained. We test the performance of several branch predictors, and the results are shown in Figure 9. It can be seen that the performance results of different branch predictors do not vary much because random branches in MC Transport program provide little opportunity for optimization. Therefore, choosing the most straightforward Bimode predictor here can meet the requirements of minimalism.

c) *Storage Hierarchy*: Figure 10 shows the hits of the L1 cache and L2 cache in the process of circular tracking particle transport. Both of them significantly affect performance, so it is necessary to maintain an appropriate L2 cache level in the particle tracking phase.

Table IV shows the impact of whether L1 cache and L2 cache are configured with prefetchers on the performance of Quicksilver code, and the data reflects the ratio of running time of different configuration programs. It can be seen that data prefetching in the L2 cache is beneficial to performance, which is in line with the expectation that data prefetching improves cache utilization spatial locality. However, L1 data prefetching has the opposite effect because MC Transport program executes more random branches from a fine granularity point of view and then causes failure after prefetching. Therefore, our configuration of the cache prefetcher is also reasonable.

We explored the choice of L2 cache size in the lab. Based on the 4-core simple core configuration architecture, using 2 MB, 1 MB, 512 kB and 256 kB L2 cache sizes, we ran quicksilver on the four architectures to compare their performance, area, and power, and the results are shown in Table V. The L2 cache significantly affects that on-chip area, but the loss of corresponding energy is not apparent in the process of shrinking. The area gain reached 1.9x during size reduction, but the performance was lost by only 7%. So the L2 cache with the size of 256 kB is determined as the cache configuration.

TABLE V
PERFORMANCE, AREA, POWER DIFFERENCES FOR CACHE SIZES

L2 Cache Size	Performance(s)	Area(mm2)	Power(mW)
2MB	2.69	9.16	80.6
1MB	2.74	6.9	78.8
512kB	2.82	5.7	77.5
256kB	2.88	4.85	76

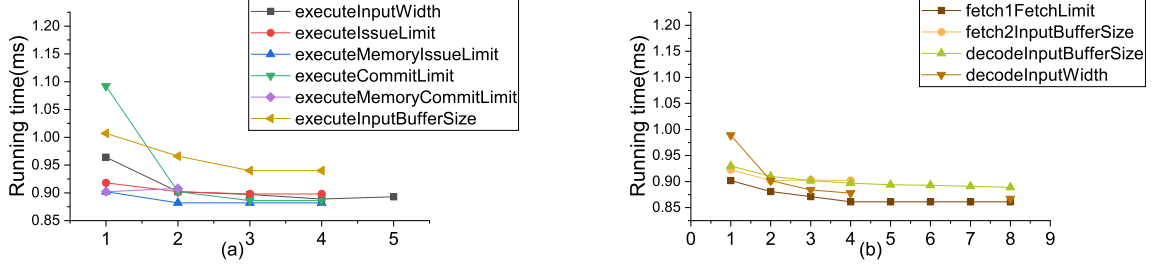


Fig. 6. Effect of adjusting pipeline structure parameters on the performance of the DAXPY.

B. Performance Degradation

The previous experiments confirmed the various structures and configurations of the MiniMCTAD architecture, and we verified the performance degradation compared with the A15 out-of-order core. According to the experimental setup, we used gem5 to model the A15 out-of-order architecture, which represents a big high-performance core, and the MiniMCTAD architecture embodies an architectural prototype that simplifies the design for such problems. We intercepted each particle tracking in the loop as a region of interest (ROI) for the running Quicksilver code and intercepted 180 trackings as performance comparison data. Figure 11 shows a comparison of the performance of the two architectures running MC Transport code.

MiniMCTAD architectures generally take more time to perform particle tracking operations, but there is no significant relationship between each tracking due to the randomness of the MC code branch control flow. According to statistics, on the whole, due to the adjustment of out-of-order instruction issuance to in-order instruction issuance, the reduction of execution units and the simplification of the issuance width and buffer size of each instruction stage, the MiniMCTAD architecture will obtain a 41% performance reduction compared with the big high-performance core. It is gratifying to note that

this figure is entirely acceptable. As noted in [18], when tested with a generic workload, out-of-order cores gained about 3x Performance advantages over in-order cores. Our experiments are precise because of the unique complex branches of MC code that the out-of-order core is often frustrated in the branch prediction process and cannot give full play to its advantages. In this case, our focus begins to focus on the power and area advantages of MiniMCTAD architecture.

C. Power and Area Gains

According to the architecture configuration of gem5 modelling and the statistical output of the running simulation, we model the A15 complex core and the 4-core MiniMCTAD core processor in McPAT. The power consumption and area comparison of the two is shown in Figure 14.

After a series of simplifications described earlier, the area advantage is 6.26x, and the power advantage is 3.92x. In this case, combined with the previous performance degradation, the MiniMCTAD architecture will also gain a 4.45x advantage on performance per watt and 2.78x advantage on performance per area.

Comparing the on-chip area of the two CPUs in Figure 13, we can see:

The overall simplification of pipeline components brings about the overall reduction of each component in the pipeline. The abandonment of out-of-order execution makes us no longer need to rename list elements, and the removal of

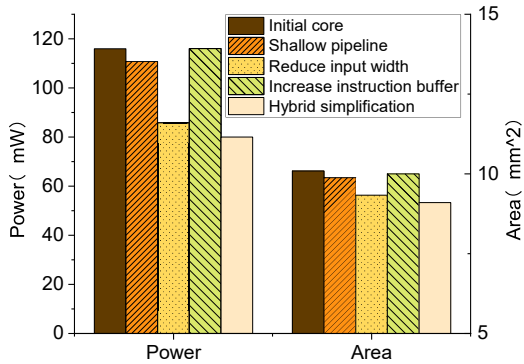


Fig. 7. Effect of adjusting pipeline structure parameters on the area and power of Quicksilver.

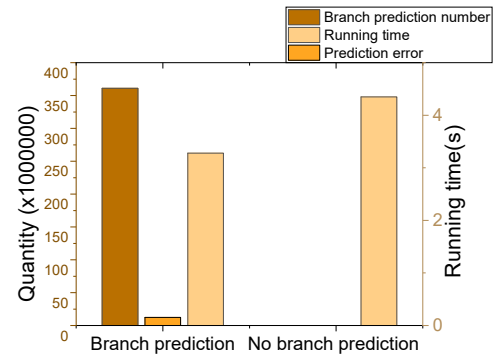


Fig. 8. The difference with or without branch prediction.

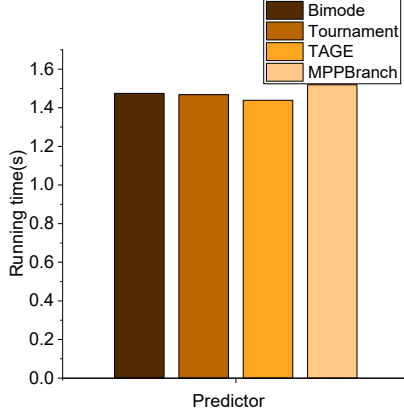


Fig. 9. Performance impact of architecture using different branch predictors.

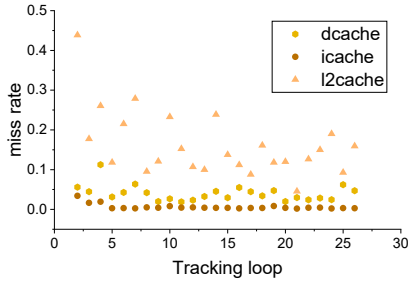


Fig. 10. Cache hits when looping particle tracking.

redundancy of execution components also brings apparent effects.

D. Exploit of Parallelism

In order to further exploit parallelism for performance advantages, we continue to model the 8-core MiniMCTAD architecture in gem5 and expand the L2 cache area to 512kB accordingly. More processors provide higher parallelism when running particle transport on the same scale, and Figure 12 shows performance tracking. Each piece of data still comes from tracking a single particle with a single core, but the increase in parallelism allows the same scale to be done in half the time. Figure 14 also shows the performance, power, and area comparison of the 8-core Extended Architecture with the previous two architectures.

Unlike the previous comparison between the MiniMCTAD core and the out-of-order core, the two MiniMCTAD architectures with only different number of cores have the same single-core performance, so due to twice the parallelism, they will gain a 29% performance advantage over the A15 architecture when performing tasks of the same size. In terms of area and power consumption, the 8-core architecture has an area of $8.92mm^2$ and an energy consumption of 148mW, respectively, which is slightly less than twice the area and power consumption of the 4-core architecture. Compared with

the A15 complex architecture, the 8-core architecture achieves a 3.22x Power consumption advantage and a 2.13x area advantage. After complete performance results, the 8-core architecture achieves a 4.56x advantage on performance per watt and 3.01x advantage on performance per area, which is slightly larger than that under the condition of an equal number of cores. Overall, many-core integration can still maintain or even achieve better performance per area performance and performance per watt.

VI. RELATED WORK

The research related to the MC method has a long history. Since MC methods generally have similar characteristics, the relevant work here is not limited to MC Transport applications, and any simulation using MC methods, including dynamic simulation and option pricing, is considered. The parallelization of the MC method is an old topic. Recently, Wang et al.'s [19] vectorized parallelization of photonic simulation application, MCML, and obtained 345x speedup compared with that before vectorization acceleration. More work has focused on the algorithm level, Dybeck et al. [20] accelerated the dynamic Monte Carlo by automatically scaling the intrinsic constant rate of the quasi-equilibrium process, and Gao et al. [21] have also demonstrated a similar hybrid scaling algorithm optimization. Liu et al. [22] studied the scanning scheduling algorithm in particle transport on Tianhe-2. Leppanen et al. [23] accelerated the convergence of fission sources in MC Transport program, while Kowalski et al. [24] optimized the cache strategy of distance data so that they can be used repeatedly in nearby tracking. In addition, FPGA acceleration is also a direction of MC acceleration [25], [26], but the long design cycle of FPGA and the high requirements for designers restrict the general use of this method. In the hardware details, the research of random number generator is the critical content of MC simulation acceleration, and the true random number generator designed by Qu et al. [27] aims at this problem. There is not much work related to the innovative design of architecture accelerated by MC. Brugger et al. [28] have realized the hybrid acceleration of CPU and FPGA, while Zhang et al. [29] have realized the heterogeneous hybrid acceleration of CPU and GPUs. The performance of the former is two orders of magnitude faster than that of a high-end Intel CPU, but the shortage of FPGA still limits the applicability.

VII. CONCLUSION

This paper provides an architecture exploration and optimization method and uses it to design a computing architecture for MC Transport applications. Using the gem5 simulator and McPAT, we explored designing a domain-specific architecture for MC Transport applications. We discuss how to design and simplify the pipeline structure, functional unit, and cache level configuration to fully use programs' inherent parallelism. Our analysis shows that high parallel programs, represented by MC Transport applications, require more parallelism than single-threaded performance. By simplifying the single-threaded architecture, although we give up single-core performance, we

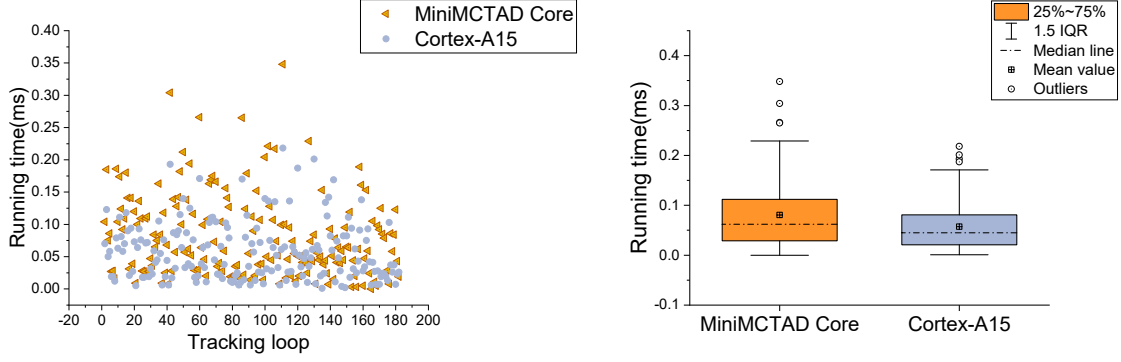


Fig. 11. Performance loss of MiniMCTAD cores compared to out-of-order cores.

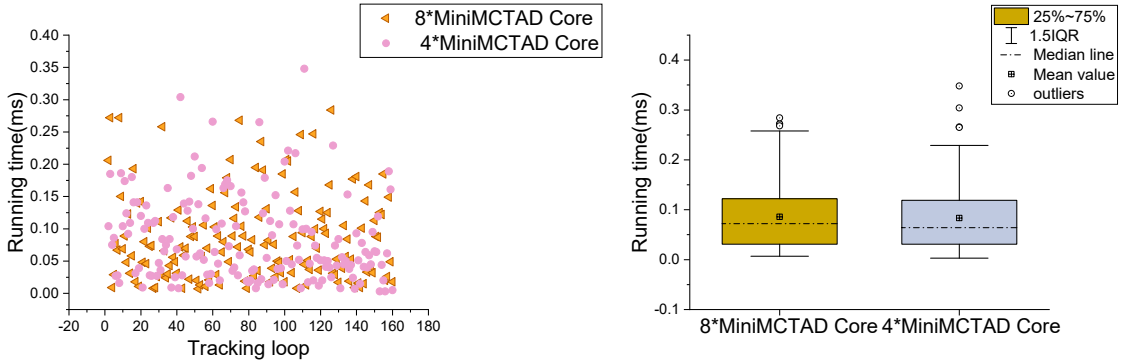


Fig. 12. Performance comparison of 4 MiniMCTAD cores compared to 8 MiniMCTAD cores.

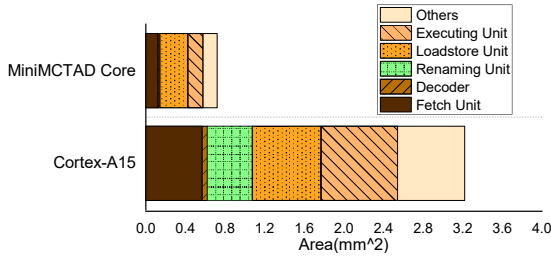


Fig. 13. Area advantage of MiniMCTAD cores compared to out-of-order cores.

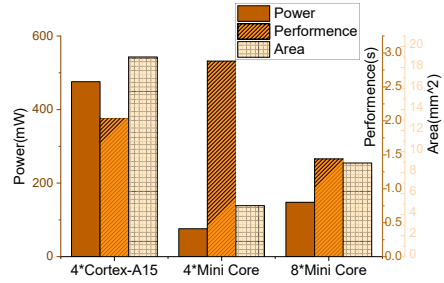


Fig. 14. Performance, area and power consumption comparison between A15 complex core and 4-core MiniMCTAD core.

gain performance, area, and power benefits by increasing parallelism, resulting in a 4.45x advantage on performance per watt and 2.78x advantage on performance per area. With the development of processor performance towards domain specialization, performance optimization at the architecture level will become a new field. Next, we will expand the number of cores integrated on a single chip to 1000 cores based on technology nodes smaller than 16nm, achieving an excellent leap in speedup. Then, We will continue to explore domain-specific architecture design for MC Transport applications, combining micro-architecture, instruction set design, and accelerator.

REFERENCES

- [1] G. E. Moore, "Progress in digital integrated electronics [technical literature, copyright 1975 ieee. reprinted, with permission. technical digest. international electron devices meeting, ieee, 1975, pp. 11-13.]," *IEEE Solid-State Circuits Society Newsletter*, vol. 11, no. 3, pp. 36–37, 2006.
- [2] R. P. Colwell, "The pentium chronicles: Introduction," *Computer*, vol. 39, no. 1, pp. 49–54, 2006.
- [3] D. Koufaty and D. T. Marr, "Hyperthreading technology in the netburst microarchitecture," *IEEE Micro*, vol. 23, no. 2, pp. 56–65, 2003.
- [4] L. Chai, Q. Gao, and D. K. Panda, "Understanding the impact of multi-core architecture in cluster computing: A case study with intel dual-core system," in *Seventh IEEE international symposium on cluster computing and the grid (CCGrid'07)*. IEEE, 2007, pp. 471–478.

- [5] E. Rotem, A. Naveh, A. Ananthakrishnan, E. Weissmann, and D. Rajwan, "Power-management architecture of the intel microarchitecture code-named sandy bridge," *Ieee micro*, vol. 32, no. 2, pp. 20–27, 2012.
- [6] C. S. Anderson, J. Zhang, and M. Cornea, "Enhanced vector math support on the intel® avx-512 architecture," in *2018 IEEE 25th Symposium on Computer Arithmetic (ARITH)*. IEEE, 2018, pp. 120–124.
- [7] C. Auth, A. Aliyarukunju, M. Asoro, D. Bergstrom, V. Bhagwat, J. Birdsall, N. Bisnik, M. Buehler, V. Chikarmane, G. Ding *et al.*, "A 10nm high performance and low-power cmos technology featuring 3 rd generation finfet transistors, self-aligned quad patterning, contact over active gate and cobalt local interconnects," in *2017 IEEE International Electron Devices Meeting (IEDM)*. IEEE, 2017, pp. 29–1.
- [8] J. Dean, D. Patterson, and C. Young, "A new golden age in computer architecture: Empowering the machine-learning revolution," *IEEE Micro*, vol. 38, no. 2, pp. 21–29, 2018.
- [9] C. E. Leiserson, N. C. Thompson, J. S. Emer, B. C. Kuszmaul, B. W. Lampson, D. Sanchez, and T. B. Schardl, "There's plenty of room at the top: What will drive computer performance after moore's law?" *Science*, vol. 368, no. 6495, 2020.
- [10] T. Makimoto, "Implications of makimoto's wave," *Computer*, vol. 46, no. 12, pp. 32–37, 2013.
- [11] X. Yang, Z. Lu, M. Qiu, and Y. Hu, "A parallel integer relative robust mean-covar model for portfolio optimization," in *2020 IEEE 6th Intl Conference on Big Data Security on Cloud (BigDataSecurity), IEEE Intl Conference on High Performance and Smart Computing (HPSC) and IEEE Intl Conference on Intelligent Data and Security (IDS)*. IEEE, 2020, pp. 180–185.
- [12] P. Brantley and M. McKinley, "Mercury web site," *Lawrence Livermore National Laboratory*, <https://mercury.llnl.gov>, 2011.
- [13] D. F. Richards, R. C. Bleile, P. S. Brantley, S. A. Dawson, M. S. McKinley, and M. J. O'Brien, "Quicksilver: a proxy app for the monte carlo transport code mercury," in *2017 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 2017, pp. 866–873.
- [14] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti *et al.*, "The gem5 simulator," *ACM SIGARCH computer architecture news*, vol. 39, no. 2, pp. 1–7, 2011.
- [15] S. Li, J. H. Ahn, R. D. Strong, J. B. Brockman, D. M. Tullsen, and N. P. Jouppi, "Mcpat: An integrated power, area, and timing modeling framework for multicore and manycore architectures," in *Proceedings of the 42nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2009, pp. 469–480.
- [16] Y. M. Qureshi, J. M. Herruzo, M. Zapater, K. Olcoz, S. G. Navarro, O. Plata, and D. Aienza, "Genome sequence alignment-design space exploration for optimal performance and energy architectures," *IEEE Transactions on Computers*, 2020.
- [17] Y. Guo, Q. Zhuge, J. Hu, J. Yi, M. Qiu, and E. H.-M. Sha, "Data placement and duplication for embedded multicore systems with scratch pad memory," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 32, no. 6, pp. 809–817, 2013.
- [18] F. A. Endo, D. Couroussé, and H.-P. Charles, "Micro-architectural simulation of embedded core heterogeneity with gem5 and mcpat," in *Proceedings of the 2015 Workshop on Rapid Simulation and Performance Evaluation: Methods and Tools*, 2015, pp. 1–6.
- [19] P.-J. Wang, C.-Y. Liu, C.-H. Tu, C.-P. Lee, and S.-H. Hung, "Acceleration of monte-carlo simulation on high performance computing platforms," in *Proceedings of the 2018 Conference on Research in Adaptive and Convergent Systems*, 2018, pp. 225–230.
- [20] E. C. Dybeck, C. P. Plaisance, and M. Neurock, "Generalized temporal acceleration scheme for kinetic monte carlo simulations of surface catalytic processes by scaling the rates of fast reactions," *Journal of chemical theory and computation*, vol. 13, no. 4, pp. 1525–1538, 2017.
- [21] H. Gao, L. J. Broadbelt, I. A. Konstantinov, and S. G. Arturo, "Acceleration of kinetic monte carlo simulations of free radical copolymerization: A hybrid approach with scaling," *AIChE Journal*, vol. 63, no. 9, pp. 4013–4021, 2017.
- [22] J. Liu, C. Lihua, W. Qing Lin, G. Chunye, J. Jie, G. Xinbiao, L. Shengguo, Q. Hu, and T. Masterson, "Parallel sn sweep scheduling algorithm on unstructured grids for multigroup time-dependent particle transport equations," *Nuclear Science and Engineering*, vol. 184, no. 4, pp. 527–536, 2016.
- [23] J. Leppänen, "Acceleration of fission source convergence in the serpent 2 monte carlo code using a response matrix based solution for the initial source distribution," *Annals of Nuclear Energy*, vol. 128, pp. 63–68, 2019.
- [24] M. A. Kowalski and P. M. Cosgrove, "Acceleration of surface tracking in monte carlo transport via distance caching," *Annals of Nuclear Energy*, vol. 152, p. 108002, 2021.
- [25] C. De Schryver, I. Shcherbakov, F. Kienle, N. Wehn, H. Marxen, A. Kostiuk, and R. Korn, "An energy efficient fpga accelerator for monte carlo option pricing with the heston model," in *2011 International Conference on Reconfigurable Computing and FPGAs*. IEEE, 2011, pp. 468–474.
- [26] L. Ma, F. B. Muslim, and L. Lavagno, "High performance and low power monte carlo methods to option pricing models via high level design and synthesis," in *2016 European Modelling Symposium (EMS)*. IEEE, 2016, pp. 157–162.
- [27] Y. Qu, J. Han, B. F. Cockburn, W. Pedrycz, Y. Zhang, and W. Zhao, "A true random number generator based on parallel stt-mtjs," in *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017*. IEEE, 2017, pp. 606–609.
- [28] C. Brugger, C. de Schryver, and N. Wehn, "Hyper: A runtime reconfigurable architecture for monte carlo option pricing in the heston model," in *2014 24th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 2014, pp. 1–8.
- [29] S. Zhang, Z. Wang, Y. Peng, B. Schmidt, and W. Liu, "Mapping of option pricing algorithms onto heterogeneous many-core architectures," *The Journal of Supercomputing*, vol. 73, no. 9, pp. 3715–3737, 2017.